1. PROJECT OVERVIEW

TransitOps is a full-stack Smart Transport Operations Platform designed to manage the complete lifecycle of transport and logistics operations.

The system provides centralized management for vehicles, drivers, trips, maintenance, fuel consumption, operational expenses, and business analytics.

The platform supports multiple user roles, including Fleet Manager, Driver/Dispatcher, Safety Officer, and Financial Analyst.

The main goal of TransitOps is to replace manual spreadsheets and logbooks with a secure, automated, and user-friendly web application.

The system includes Role-Based Access Control (RBAC), automatic vehicle and driver status management, business rule validations, dashboard KPIs, reports, analytics, and export functionality.

2. FRONTEND TECHNOLOGIES

React.js

React.js will be used to build the frontend user interface of the TransitOps platform.

React allows us to create reusable components such as Header, Sidebar, Dashboard Cards, Tables, Forms, Modals, Charts, and Filters.

The frontend will contain pages for Dashboard, Vehicle Management, Driver Management, Trip Management, Maintenance, Fuel Logs, Expenses, Reports, and User Management.

TypeScript

TypeScript will be used with React to improve code quality and reduce runtime errors.

TypeScript provides static type checking for API responses, component properties, forms, vehicles, drivers, trips, and other application data.

Vite

Vite will be used as the frontend development and build tool.

It provides fast project startup, Hot Module Replacement (HMR), optimized production builds, and a simple development environment.

React Router DOM

React Router DOM will be used for navigation and routing between different pages.

It will also help implement Protected Routes and Role-Based Routes.

Example routes:

/login

/dashboard

/vehicles

/drivers

/trips

/maintenance

/fuel-logs

/expenses

/reports

/users

Bootstrap 5

Bootstrap 5 will be used as the main CSS framework.

It will help create responsive layouts, forms, tables, buttons, modals, dropdowns, navigation components, and utility classes.

Custom CSS

Custom CSS will be used with Bootstrap to create the professional design and branding of TransitOps.

Custom CSS will handle dashboard styling, sidebar design, responsive behavior, animations, status badges, spacing, and other UI improvements.

Axios

Axios will be used to communicate between the React frontend and backend REST APIs.

Axios will handle GET, POST, PUT, PATCH, and DELETE requests.

Axios Interceptors can be used to automatically attach JWT authentication tokens to protected API requests.

React Hook Form

React Hook Form will be used to manage application forms.

It will be useful for Vehicle Registration, Driver Registration, Trip Creation, Maintenance Records, Fuel Logs, Expense Records, Login, and User Management forms.

Zod

Zod will be used for frontend form validation.

It will help validate data such as email addresses, required fields, positive numbers, cargo weight, vehicle capacity, dates, and other input values.

Recharts

Recharts will be used to display charts and visual analytics on the dashboard and reports pages.

Examples:

Fuel Consumption Charts

Fleet Utilization Charts

Operational Cost Charts

Vehicle ROI Charts

Maintenance Cost Charts

Trip Status Charts

3. BACKEND TECHNOLOGIES

Node.js

Node.js will be used as the backend runtime environment.

It will execute the server-side application logic and handle API requests.

Express.js

Express.js will be used to build the REST API.

The backend will contain APIs for:

Authentication

Users

Roles

Vehicles

Drivers

Trips

Maintenance

Fuel Logs

Expenses

Dashboard

Reports

File Uploads

TypeScript

TypeScript will also be used in the backend.

It will provide better type safety, maintainability, scalability, and development experience.

REST API

REST API architecture will be used for communication between frontend and backend.

Example APIs:

POST /api/auth/login

GET /api/vehicles

POST /api/vehicles

PUT /api/vehicles/:id

DELETE /api/vehicles/:id

GET /api/drivers

POST /api/trips

PATCH /api/trips/:id/dispatch

PATCH /api/trips/:id/complete

POST /api/maintenance

GET /api/dashboard/kpis

GET /api/reports/fleet-utilization

4. DATABASE TECHNOLOGIES

MySQL

MySQL will be used as the relational database management system.

A relational database is suitable for TransitOps because the application contains strongly related data such as users, roles, vehicles, drivers, trips, maintenance records, fuel logs, and expenses.

Main Database Tables:

Users

Roles

Vehicles

Drivers

Trips

MaintenanceLogs

FuelLogs

Expenses

VehicleDocuments

RefreshTokens

Prisma ORM

Prisma ORM will be used to communicate between the Node.js backend and MySQL database.

Prisma provides database models, migrations, relationships, type-safe queries, and easier database management.

5. AUTHENTICATION AND AUTHORIZATION

JWT (JSON Web Token)

JWT will be used for authentication.

After successful login, the backend will generate an Access Token.

The token will be sent with protected API requests.

Refresh Token

Refresh Tokens can be implemented to provide better authentication security and user experience.

The Access Token will have a short expiration time, while the Refresh Token will be used to generate a new Access Token.

bcrypt

bcrypt will be used to securely hash user passwords before storing them in the database.

Plain-text passwords will never be stored in the database.

Role-Based Access Control (RBAC)

RBAC will control which features users can access.

Example:

Fleet Manager:

Manage Vehicles

Manage Maintenance

View Fleet Analytics

Driver / Dispatcher:

Create Trips

Dispatch Trips

Complete Trips

View Available Vehicles and Drivers

Safety Officer:

Manage Driver Compliance

Monitor License Expiration

Monitor Safety Scores

Financial Analyst:

Manage Expenses

View Fuel Costs

View Maintenance Costs

View Vehicle ROI

View Financial Reports

6. BUSINESS RULE VALIDATION

Backend Services and Database Transactions will be used to implement important business rules.

The system will validate that:

Vehicle Registration Number is unique.

Retired vehicles cannot be assigned to trips.

Vehicles In Shop cannot be assigned to trips.

Vehicles already On Trip cannot be assigned to another trip.

Drivers with expired licenses cannot be assigned to trips.

Suspended drivers cannot be assigned to trips.

Drivers already On Trip cannot be assigned to another trip.

Cargo Weight cannot exceed Vehicle Maximum Load Capacity.

Dispatching a trip changes Vehicle and Driver status to On Trip.

Completing a trip changes Vehicle and Driver status to Available.

Cancelling a dispatched trip restores Vehicle and Driver status to Available.

Creating active maintenance changes Vehicle status to In Shop.

Closing maintenance changes Vehicle status back to Available unless the vehicle is Retired.

Database Transactions will be used when multiple database records must be updated together.

7. DASHBOARD AND ANALYTICS

The TransitOps dashboard will display important operational KPIs.

Dashboard KPIs:

Active Vehicles

Available Vehicles

Vehicles In Maintenance

Active Trips

Pending Trips

Drivers On Duty

Fleet Utilization Percentage

Total Fuel Cost

Total Maintenance Cost

Total Operational Cost

Analytics:

Fuel Efficiency

Distance / Fuel Consumed

Fleet Utilization

Active Vehicles / Total Vehicles × 100

Operational Cost

Fuel Cost + Maintenance Cost

Vehicle ROI

(Revenue - Fuel Cost - Maintenance Cost) / Acquisition Cost

8. FILE UPLOAD AND DOCUMENT MANAGEMENT

Multer

Multer will be used in the backend to receive uploaded files.

Cloudinary

Cloudinary can be used to store vehicle documents and other uploaded files.

The database will store the file URL and document information.

Vehicle documents may include:

Registration Certificate

Insurance Document

Pollution Certificate

Permit

Other Vehicle Documents

9. EMAIL NOTIFICATION SYSTEM

Nodemailer

Nodemailer will be used to send email notifications.

It can send reminders when driver licenses are close to expiration.

node-cron

node-cron will be used to run scheduled background jobs.

For example, every day the system can check for driver licenses expiring within the next 30 days and send reminder emails.

10. CSV EXPORT

json2csv

json2csv will be used to convert report data into CSV format.

Users can download reports such as:

Vehicle Reports

Driver Reports

Trip Reports

Fuel Reports

Expense Reports

Operational Cost Reports

11. PDF EXPORT

PDFKit

PDFKit can be used to generate PDF reports from the backend.

PDF export is an optional bonus feature of the TransitOps platform.

12. SECURITY TECHNOLOGIES

Helmet

Helmet will be used to add secure HTTP headers to the Express application.

CORS

CORS middleware will control which frontend applications can access the backend API.

express-rate-limit

express-rate-limit will protect authentication APIs from excessive requests and brute-force login attempts.

bcrypt

bcrypt will protect user passwords using secure password hashing.

JWT Authentication Middleware

JWT middleware will verify users before allowing access to protected APIs.

RBAC Middleware

RBAC middleware will check whether users have permission to access specific APIs.

13. API TESTING

Postman

Postman will be used to test the backend REST APIs.

It will be used to test authentication, CRUD operations, business validations, authorization, and API responses.

14. VERSION CONTROL

Git

Git will be used for source code version control.

GitHub

GitHub will be used to store the project repository, collaborate with team members, manage branches, and maintain project history.

Recommended Branch Structure:

main

develop

feature/authentication

feature/dashboard

feature/vehicle-management

feature/driver-management

feature/trip-management

feature/maintenance

feature/expense-management

15. DEVELOPMENT TOOLS

Visual Studio Code or JetBrains Rider

Code Editor / IDE

MySQL Workbench

Database Management

Postman

API Testing

Git and GitHub

Version Control

npm

Package Management

ESLint

Code Quality and Error Detection

Prettier

Code Formatting

16. DEPLOYMENT TECHNOLOGIES

Vercel

Vercel can be used to deploy the React frontend.

Render or Railway

Render or Railway can be used to deploy the Node.js and Express backend.

Railway MySQL or Aiven MySQL

A cloud-hosted MySQL database can be used for production deployment.

Cloudinary

Cloudinary can be used for cloud file storage.

17. COMPLETE TECHNOLOGY STACK SUMMARY

Frontend:

React.js

TypeScript

Vite

React Router DOM

Bootstrap 5

Custom CSS

Axios

React Hook Form

Zod

Recharts

Backend:

Node.js

Express.js

TypeScript

REST API

Database:

MySQL

Prisma ORM

Authentication and Security:

JWT

Refresh Token

bcrypt

RBAC

Helmet

CORS

express-rate-limit

File Management:

Multer

Cloudinary

Notifications:

Nodemailer

node-cron

Reports and Export:

Recharts

json2csv

PDFKit

Testing:

Postman

Development and Version Control:

Git

GitHub

ESLint

Prettier

Deployment:

Vercel

Render or Railway

Cloud MySQL Database

Cloudinary

18. FINAL PROJECT ARCHITECTURE

USER

↓

REACT + TYPESCRIPT FRONTEND

↓

AXIOS

↓

REST API

↓

NODE.JS + EXPRESS.JS + TYPESCRIPT BACKEND

↓

AUTHENTICATION + RBAC + BUSINESS RULE SERVICES

↓

PRISMA ORM

↓

MYSQL DATABASE

Supporting Services:

CLOUDINARY → Vehicle Document Storage

NODEMAILER + NODE-CRON → License Expiry Notifications

RECHARTS → Dashboard and Analytics

JSON2CSV → CSV Report Export

PDFKIT → PDF Report Export

GIT + GITHUB → Version Control

FINAL RECOMMENDED STACK:

React.js + TypeScript + Vite + Bootstrap 5 + Axios + React Hook Form + Zod + Recharts

Node.js + Express.js + TypeScript + REST API

MySQL + Prisma ORM

JWT + Refresh Tokens + bcrypt + RBAC

Multer + Cloudinary

Nodemailer + node-cron

json2csv + PDFKit

Git + GitHub

Vercel + Render/Railway + Cloud MySQL Database